GIT Y GITHUB: ADMINISTRACIÓN DE REPOSITORIOS REMOTOS

Institución educativa: Fundación universitaria Agraria de Colombia

Programa académico: Medicina Veterinaria

Asignatura: Introducción a la ciencia de datos

Título del trabajo: Tutorial de Git — Capítulo 3: Git y GitHub

Estudiante: July Manuela Romero Romero 37973

Docente: Carlos Andres Gomez

1. Introducción

Git permite administrar el historial de un proyecto en una computadora local, mientras que GitHub ofrece un servicio para alojar repositorios Git en Internet. La combinación de ambas herramientas facilita conservar una copia remota del trabajo y compartir los cambios realizados.

Para utilizar este flujo de trabajo es necesario crear un repositorio en GitHub, obtener su dirección, clonar el repositorio en la computadora y enviar las confirmaciones locales al servidor remoto.

Este capítulo explica cómo conectar un repositorio local con GitHub y utilizar comandos como git clone, git remote, git push y git pull. También se presenta un ejercicio práctico para publicar los capítulos de un tutorial.

2. Objetivos

2.1. Objetivo general

Comprender y aplicar los procedimientos necesarios para sincronizar un repositorio Git local con un repositorio remoto en GitHub.

2.2. Objetivos específicos

- Crear un repositorio remoto en GitHub.
- Clonar el repositorio en la computadora.
- Identificar la función de los repositorios remotos.
- Enviar confirmaciones mediante git push.
- Obtener cambios remotos mediante git pull.
- Documentar el proceso de publicación de archivos.

3. Desarrollo teórico

3.1. ¿Qué es GitHub?

GitHub es una plataforma de alojamiento de repositorios Git. Permite guardar proyectos, consultar el historial, revisar archivos y compartir el trabajo con otras personas.

Un repositorio puede configurarse como público o privado, de acuerdo con las necesidades del usuario y las instrucciones de la actividad. Para un trabajo académico individual, el estudiante debe mantener el control de su propio repositorio y verificar que los archivos sean accesibles para el docente cuando corresponda.

3.2. Crear un repositorio en GitHub

El procedimiento general es el siguiente:

1. Iniciar sesión en https://github.com.
2. Seleccionar la opción para crear un repositorio nuevo.
3. Asignar un nombre, por ejemplo tutorial-git.
4. Agregar una descripción del proyecto.
5. Elegir la visibilidad pública o privada.
6. Crear el repositorio.
7. Copiar la URL HTTPS que proporciona GitHub.

La dirección tendrá una estructura semejante a esta:

https://github.com/USUARIO/tutorial-git.git

La URL es un ejemplo y debe reemplazarse por la dirección real del repositorio.

3.3. Clonar el repositorio

Clonar significa descargar una copia del repositorio remoto y su historial para trabajar localmente.

git clone https://github.com/USUARIO/tutorial-git.git

cd tutorial-git

El primer comando crea una carpeta con el contenido del repositorio. El segundo permite entrar en esa carpeta desde la terminal.

Si se clona un repositorio que ya contiene un archivo README, no es necesario crear de nuevo ese archivo.

3.4. Repositorios remotos

Un remoto es una referencia a otro repositorio Git. Cuando se clona un repositorio, normalmente se configura automáticamente un remoto llamado origin.

Para consultar los remotos configurados se utiliza:

git remote -v

El resultado muestra las direcciones asociadas con las operaciones de lectura y envío.

Si se necesita agregar un remoto manualmente, se utiliza:

git remote add origin https://github.com/USUARIO/tutorial-git.git

Este último comando solo debe ejecutarse si el remoto todavía no existe. Si origin ya está configurado, no se debe agregar otra vez.

3.5. Enviar cambios con git push

El comando git push envía al repositorio remoto las confirmaciones locales que correspondan a la rama configurada.

Un flujo de trabajo habitual es:

git status

git add .

git commit -m "docs: agregar capitulo 1"

git push

En el primer envío de una rama puede ser necesario indicar la rama remota:

git push -u origin main

En este ejemplo se utiliza la rama main. Si el repositorio trabaja con otra rama, se debe utilizar el nombre correspondiente.

El comando git push no guarda por sí mismo los cambios que todavía no han sido confirmados. Por eso, primero se preparan y confirman los archivos.

3.6. Obtener cambios con git pull

El comando git pull obtiene los cambios del repositorio remoto y los integra en la rama local actual, mediante el mecanismo de integración configurado.

git pull

Es útil cuando se han realizado modificaciones en GitHub o cuando otra copia del proyecto ha enviado nuevos cambios.

Antes de comenzar a trabajar, conviene revisar el estado local para evitar conflictos. Si Git detecta cambios incompatibles, puede ser necesario resolverlos antes de continuar.

3.7. Autenticación y seguridad

GitHub puede solicitar autenticación al intentar enviar cambios. Se deben seguir los mecanismos admitidos por la plataforma, como un administrador de credenciales, autenticación mediante navegador o claves SSH configuradas correctamente.

No se deben escribir contraseñas, tokens de acceso ni claves privadas dentro de los documentos del proyecto. Estos datos deben mantenerse protegidos.

4. Ejercicio práctico: publicar los capítulos

Se propone trabajar en un repositorio individual llamado tutorial-git, con una estructura semejante a esta:

tutorial-git/

├── README.md

├── practica.txt

├── Capitulo1.docx

├── Capitulo1.pdf

├── Capitulo2.docx

├── Capitulo2.pdf

├── Capitulo3.docx

└── Capitulo3.pdf

Los archivos deben existir realmente en la carpeta local antes de ejecutar los comandos. La estructura es una propuesta que puede ajustarse a la organización elegida.

4.1. Publicar el primer capítulo

Después de guardar el primer documento Word y su versión PDF en la carpeta del repositorio:

git status

git add Capitulo1.docx Capitulo1.pdf

git commit -m "docs: agregar capitulo 1 de Git"

git push

4.2. Publicar el segundo capítulo

git status

git add Capitulo2.docx Capitulo2.pdf

git commit -m "docs: agregar capitulo 2 de Git"

git push

4.3. Publicar el tercer capítulo

git status

git add Capitulo3.docx Capitulo3.pdf

git commit -m "docs: agregar capitulo 3 de Git"

git push

4.4. Verificar el resultado

Después de cada envío, se debe abrir el repositorio en GitHub y comprobar que los archivos aparezcan correctamente. También se puede ejecutar:

git log --oneline

git remote -v

git status

Estas consultas permiten revisar el historial, las direcciones remotas y el estado local del repositorio.

5. Importancia del archivo README.md

El archivo README.md sirve como presentación y guía del repositorio. Debe explicar el propósito del proyecto, enumerar los documentos incluidos, describir el procedimiento realizado y mencionar las herramientas utilizadas.

Es recomendable actualizarlo cuando se agrega un capítulo nuevo y registrar esa actualización mediante un commit descriptivo.

6. Conclusiones

La integración de Git y GitHub permite mantener una copia remota del trabajo y conservar un historial de modificaciones. El uso adecuado de git clone, git remote, git push y git pull facilita la sincronización entre el repositorio local y el remoto.

La publicación de cada capítulo mediante confirmaciones individuales permite identificar claramente los avances de la actividad. Finalmente, el archivo README complementa el trabajo al explicar su organización, objetivos y procedimiento.

7. Bibliografía

Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.), capítulo 2: Fundamentos de Git. Apress. https://git-scm.com/book/es/v2/Fundamentos-de-Git

Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.), capítulo 6: GitHub. Apress. https://git-scm.com/book/es/v2/GitHub

GitHub. (s. f.). *Documentación de GitHub*. https://docs.github.com/es

Git. (s. f.). *Documentación oficial de Git*. https://git-scm.com/doc